BRAINSMINGLE AI SUMMIT 2026 · AI IN LEARNING & DEVELOPMENT · MON 12 OCT 2026 · 8:00PM CAIRO TIME

AI-Era Software Engineering

From Graduate Readiness to Talent Development

Mon 12 Oct 2026 · 8:00PM Cairo time

WHAT WE'LL COVER

01
The shift
From typing to directing

02
Universities & educators
Keep, shift, add, prove

03
Students
Learn with AI, prove it without AI

04
Fresh graduates & juniors
The new entry-level bar

05
Experienced software engineers
Grow judgment, not just reach

06
Software companies & teams
A 90-day capability

07
Software buyers & stakeholders
Faster — not free

Hany Saad
Senior Software Engineering Manager · ITWorx

20+ years in software engineering — building, architecting and leading software engineering and AI teams. Roles include Senior Software Engineering Manager at ITWorx and Software Engineering & GenAI Expert at ITI and MCIT, alongside other previous roles. Delivered talks at local and international technology events.

Connect on LinkedIn

Follow on BrainsMingle

Illustrative · simulated

100%

shop-api · Final-year project

Built in 2 h 14 min with an AI coding agent

FILES

src/Program.cs

src/Controllers/AuthController.cs

src/Data/AppDbContext.cs

tests/AuthTests.cs

README.md

READY · Grade A

CHECKS

- Build passing
- 23 / 23 tests
- Docs generated
- Deployed

? Can the author explain it?

FINAL-YEAR PROJECT · BUILT WITH AN AI CODING AGENT

Add secure authentication to this application.

AGENT

› Reading repository…

› Investigating auth dependencies…

› Generating tests…

› Updating docs…

› Opening pull request…

✓ Implementation ready for review

The code is ready.

Are we?

✓ Implementation generated

✓ Tests generated

✓ Documentation generated

✓ Pull request opened

AI-Era Software Engineering

From Graduate Readiness to Talent Development

READY FOR WHAT?

Who’s in the room?

brainsmingle-ai-era-software-engineering.lovable.app/vote?poll=s2-challenge

Scan to vote · no sign-in · 10 seconds

Live results appear here during the talk

1. **Learning** — student or recent graduate
2. **Building** — developer or engineer
3. **Teaching** — educator, professor or trainer
4. **Leading** — tech lead, manager or company leader

Watching without a phone? Type 1, 2, 3 or 4 in chat.

THE ROUTE

Seven stops. One question.

The code is ready. Are we?



01
The shift
From typing every line to directing, reviewing and validating

02
Universities & educators
What to keep, shift and add — and how to assess

03
Students
Learn with AI without skipping the learning

04
Fresh graduates & juniors
The new entry bar and your first 90 days

05
Experienced software engineers
Grow judgment as fast as AI grows your reach

06
Software companies & teams
Adopt, train, measure: a 90-day roadmap

07
Software buyers & stakeholders
Faster — not free, and not instant

ACTION Every stop ends with one action you can start this week.

→ Closing · our answer

STOP 01

The shift

What changed when AI started writing the code?

01

01 **The shift**
02 Universities & educators
03 Students
04 Fresh graduates & juniors
05 Experienced software engineers
06 Software companies & teams
07 Software buyers & stakeholders

Everyone can build software now.

BUILDERS WHO DON'T CODE

App builders

Lovable · Bolt · v0 · Replit Agent · Google AI Studio

Workflow automation

n8n · Make · Zapier · Power Automate / Copilot Studio

Describe it → get a working app or automation.

Lovable: $200M ARR — Fortune, Nov 2025

“Everyone becomes a developer in the future.” — Anton Osika, Lovable CEO

This presentation is a Lovable app.

DEVELOPERS

AI-assisted coding — you drive

GitHub Copilot · Cursor · JetBrains AI · Gemini Code Assist

AI coding agents — you delegate

Claude Code · OpenAI Codex · Copilot coding agent · Gemini CLI · Kiro

Among coding-agent users: Claude Code 66%, GitHub Copilot 59% — Stack Overflow 2026

Sources: Fortune, Nov 2025 · Stack Overflow 2026

The unit of delegation changed.

1 **Autocomplete** — a line

2 **Chat assistant** — a question

3 **IDE copilot** — a function

4 **Repository-aware agent** — an issue

5 **Agentic workflow** — a feature loop

6 **AI-native delivery** — a workflow

Sources: GitHub, Apr 2026

THEN

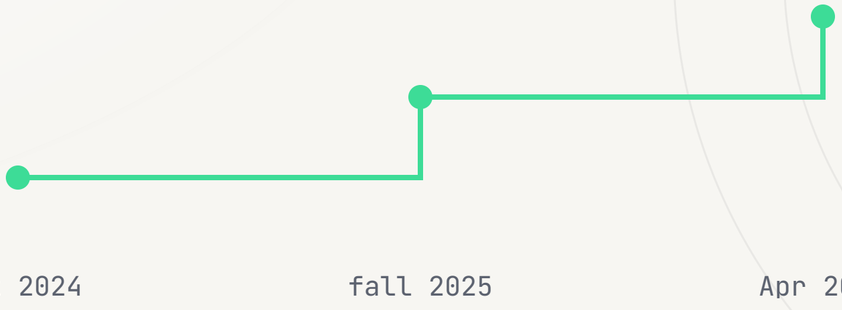
Write a C# method to validate a JWT.

NOW

Investigate the existing authentication implementation, propose a secure integration plan, identify affected components, add tests, and prepare a PR for review.

75%

of new code at Google is AI-generated · Apr 2026



> "Today, 75% of all new code at Google is now AI-generated and approved by engineers, up from 50% last fall."
>
> **Sundar Pichai** · CEO, Google & Alphabet · Cloud Next '26 · 22 Apr 2026

> 80%

of code merged at Anthropic was authored by Claude (May 2026)

Anthropic, Jun 2026

Claude Cowork: "all" of it built with Claude Code

— Boris Cherny, Anthropic · Axios, Jan 2026

"We spent more time making product and architecture decisions than writing individual lines of code" — Felix Rieseberg, Anthropic

90%

of professional developers use AI coding agents at work weekly · 68% daily

JetBrains, Aug 2026

73%

of developers who use AI coding assistants or agents use them daily

Stack Overflow, Oct 2026

75% of the code. 100% of the accountability.

Sources: Google, Oct 2024 · Apr 2026 · Anthropic, Jun 2026 · Axios, Jan 2026 · JetBrains, Aug 2026 · Stack Overflow, Oct 2026

THE SHIFT, IN THEIR WORDS

Developers increasingly direct, review and validate.

"...programming via LLM agents is increasingly becoming a default workflow for professionals, except with more oversight and scrutiny."

Andrej Karpathy
OpenAI founding member
X · 4 Feb 2026

"Our engineers are orchestrating fully autonomous digital task forces, firing off agents and accomplishing incredible things."

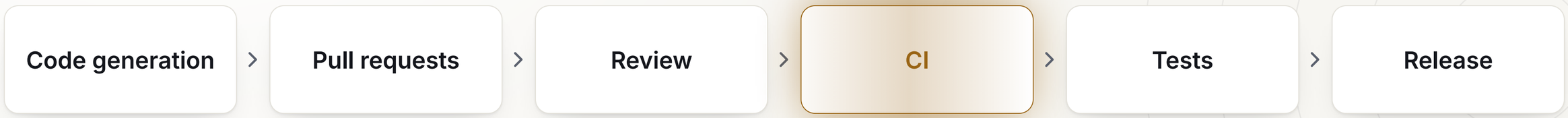
Sundar Pichai
CEO, Google & Alphabet
Cloud Next '26 · 22 Apr 2026

"These developers moved from writing code to architecting and verifying the implementation work that is carried out by AI agents."

Thomas Dohmke
then CEO of GitHub, with Eirini Kalliamvakou
Developers, Reinvented · 3 Aug 2025

THE TRUST GAP 48% trust AI when they can easily verify its answers — only 7% trust it with important decisions.

Sources: Karpathy on X, Feb 2026 · Google Cloud Next '26, Apr 2026 · Dohmke & Kalliamvakou, Aug 2025 · Stack Overflow, Oct 2026



Claude authors ~80% of code · CI jobs 25× in six months · tests 10× · test selection redesigned

Anthropic engineering blog, 14 Sep 2026 · One organisation's engineering setting

> “Writing code is no longer the constraint, and once PR review gets accelerated, CI starts feeling the pressure.”
>
> — Sachin Malhotra, Anthropic

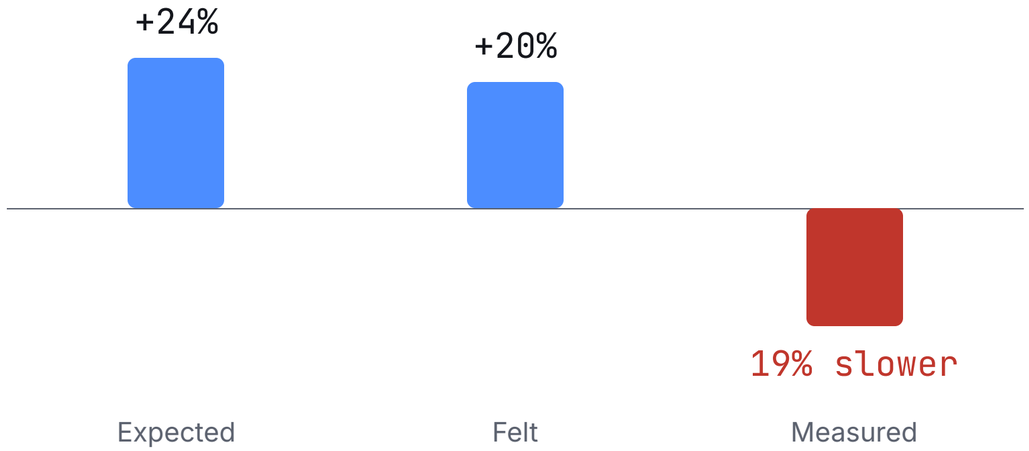
A faster implementation workflow can expose a slower delivery system.

Sources: Anthropic, 14 Sep 2026

DIFFERENT STUDIES, DIFFERENT MEASURES — NOT ONE AXIS.

METR

Jul 2025 · 16 experienced OSS developers · early-2025 tools · METR now marks it out of date



Effect on speed — measured: tasks took 19% longer with AI

Cui et al., Management Science

Field RCTs · 4,867 developers

+26%

completed tasks

Biggest gains for less-experienced developers

Faros AI, Sep 2026 (telemetry, 22,000 developers): PRs merged with no review +76% · time in QA +301%

DORA: “AI’s primary role in software development is that of an amplifier.” · Stack Overflow 2025: 66% say AI output is “almost right, but not quite.”

Code got cheaper. Being wrong didn’t.

Sources: METR, Jul 2025 · Feb 2026 · Cui et al., Management Science · Faros AI, Sep 2026 · DORA, Mar 2026 · Stack Overflow, 2025

Agentic coding isn’t ‘the agent does my work’.

THE AGENT DID THE TASKS

✓ Read the repository

✓ Edited 14 files

✓ Wrote 23 tests

✓ Opened the pull request

Only you can do the engineering

- Decide what “secure” means here
- Notice what’s missing, not just what’s wrong
- Say “no — not like that”
- Answer for it in production

The agent does the tasks. You do the engineering.

YOUR MOVE · EVERYONE, THIS WEEK

Never delegate what you can’t verify.

1

Give an agent one real task

Bounded, in a real repository — not a toy demo.

2

Read every diff

Run it. Try to break it. Ask why each change is there.

3

Write the test it didn’t

The negative case: the token that should fail.

4

Log what it got wrong

That list is your learning plan.

> 66% of developers say their biggest frustration is AI output that is “almost right, but not quite.” — Stack Overflow Developer Survey 2025

ACTION **This week: one real task, one agent — and proof before you ship.**

Sources: Stack Overflow, 2025

STOP 02

Universities & educators

What should we teach — and how should we assess it?

02

AI-native ≠ fundamentals-free.

KEEP — the core

- programming
- data structures & algorithms
- databases
- networking
- operating systems
- security
- testing
- debugging
- design

Fundamentals are how you catch what the agent got wrong.

SHIFT — the weight

- syntax → concepts
- APIs → systems
- writing → reading & evaluating
- small tasks → end-to-end problems
- one right answer → defended trade-offs
- given specs → ambiguous requirements

AI-native ≠ fundamentals-free.

ADD — the new

- agentic workflows
- context engineering
- AI code review & validation
- AI-aware security
- specification writing

PROVE — the evidence

- oral defence
- live modification
- planted-bug review
- design rationale
- process logs
- AI-off checkpoints

> **“We need to teach abstraction, decomposition, and specification not just as pre-coding steps, but as the new coding.”**
>
> — Thomas Dohmke, then CEO of GitHub, with Eirini Kalliamvakou · Aug 2025

Sources: Thomas Dohmke, Aug 2025

Assignment 3: Add secure authentication.

New requirement: accept only tokens issued for shop-api. You have 15 minutes.

WHAT WE GRADED

Compiles ✓

Tests pass ✓

Endpoints work ✓

Code style ✓

~~A+~~

WHAT WE SHOULD GRADE

~~Did the code work?~~	→ Can the student explain why it works?
~~Are tests passing?~~	→ Can they say what the tests miss?
~~Is it implemented?~~	→ Can they change it when the requirement changes?
~~Was it submitted?~~	→ Can they defend the decisions?

Anthropic redesigned its own take-home after Claude Opus 4.5 matched top candidates within 2 hours.

> “The original worked because it resembled real work. The replacement works because it simulates novel work.” — Tristan Hume, Anthropic, 21 Jan 2026

Faculty agree: 54% say current student-evaluation methods are no longer adequate — Digital Education Council, 2025

Are we developing people who can produce an answer — or engineers who can tell whether the answer is right?

Sources: Anthropic, 21 Jan 2026 · Digital Education Council, Jan 2025

YOUR ROLLOUT · UNIVERSITIES & EDUCATORS

Change one assignment this term.

This month

SET THE RULE

- An AI rule for every assignment: secure lane (AI-off, oral) or open lane (AI scaffolded)
- — or the five-level AI Assessment Scale
- Every lecturer runs an agent on their own assignment

This term

CHANGE THE EVIDENCE

- One assignment per course adds a 5-minute oral defence
- A planted-bug review and a live change request
- A process log
- One rubric, calibrated across staff

This year

CHANGE THE COURSE

- AI-off checkpoints and an agentic capstone
- A monthly staff community of practice
- Compare grades with oral-defence results
- Update the syllabus: keep, shift, add, prove

> 95% of UK undergraduates use AI — only 48% feel their teaching staff help them build AI skills (HEPI 2026). Purdue rebuilt 22 courses and 300+ plans of study for its AI-competency requirement.

ACTION **This term: change one assignment — AI allowed, explanation required.**

Sources: University of Sydney · AI Assessment Scale · HEPI, Mar 2026 · Purdue, Aug 2026

STOP 03

Students

What should you learn now — and how?

03

STUDENTS · THE MYTH

“AI does everything — so I don’t need to learn.” Wrong.

Less of

- memorising syntax
- typing boilerplate
- hunting through API docs

More of engineering

- design
- debugging
- testing
- security
- reading & reviewing code

> “Some people today are discouraging others from learning programming on the grounds AI will automate it. This advice will be seen as some of the worst career advice ever given.”

Andrew Ng
Founder, DeepLearning.AI
The Batch, Issue 292 · 12 Mar 2025

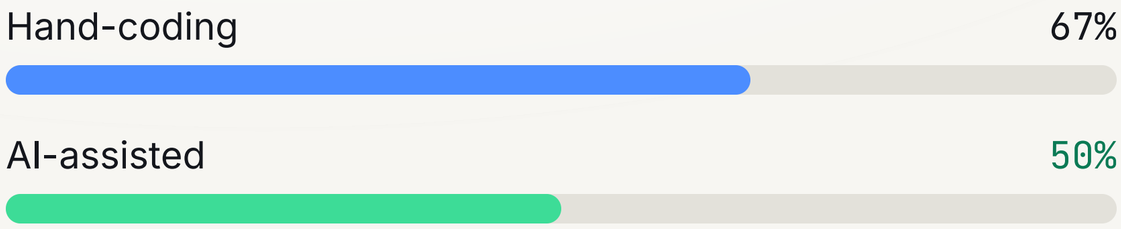
You still read and understand every line — and write it yourself when needed.

Good news: learning a new concept has never been faster. The catch: there’s more to learn.

Sources: Andrew Ng, The Batch, Mar 2025

Learn with AI — and without it.

Anthropic randomised trial · 29 Jan 2026 · 52 mostly junior engineers



`Asked AI to explain: ≥65% · Delegated to AI: <40%`

`Small pattern groups; directional`

THE LEARNING ORDER

1. Write it yourself
2. Use AI to explain, compare and critique
3. Delegate to agents — once you can verify the result

Agentic coding is a skill too — learn it after you've written code yourself.

DORA, Mar 2026 — the expertise paradox: lower barriers risk “bypassing the ‘productive struggle’ necessary for deep technical expertise.”

“That struggle is the part where the learning happens.” — Chris Gregg, Stanford

Use AI to accelerate learning — not to skip learning.

Sources: Anthropic, 29 Jan 2026 · DORA, Mar 2026 · Stanford Daily, Mar 2026

YOUR PLAN · STUDENTS

Learn with AI. Prove it without AI.

Foundations

THE ENGINEERING CORE

- Know how an OS runs your code
- Model data and write real SQL
- Design with OOP — not just classes
- Solve the problem before you prompt

See the whole SDLC: requirements → design → build → test → deploy → operate

Breadth

GO BROAD, NOT NARROW

- Be a product engineer — own a feature end to end: the user problem, UI, API, data, tests, release
- Learn a business domain (fintech, health, e-commerce…)
- Keep one deep strength (T-shaped)

Edge

WHAT SETS YOU APART

- Technical English — a prompt is a specification; write it clearly in English
- GenAI & agents — after you can code: by hand → with AI → with agents
- Habits: first version by hand · log what the AI got wrong · teach back one concept

> Lee et al. (CHI 2025): the more people trust AI, the less critical thinking they report.

EXAMPLE TOOLS — NOT ENDORSEMENTS

LEARN ChatGPT study mode · Claude learning mode · Gemini Guided Learning · CS50.ai

CODE GitHub Copilot (free for verified students) · Cursor

AGENTS Claude Code · OpenAI Codex · Gemini CLI

TEST xUnit / pytest · Playwright

ACTION **One project: build it by hand, extend it with an agent, explain every line.**

→ Graduates & juniors: all of this applies to you — Stop 04 adds the job-market reality and more tips.

Sources: Lee et al., CHI 2025 · GitHub Docs

STOP 04

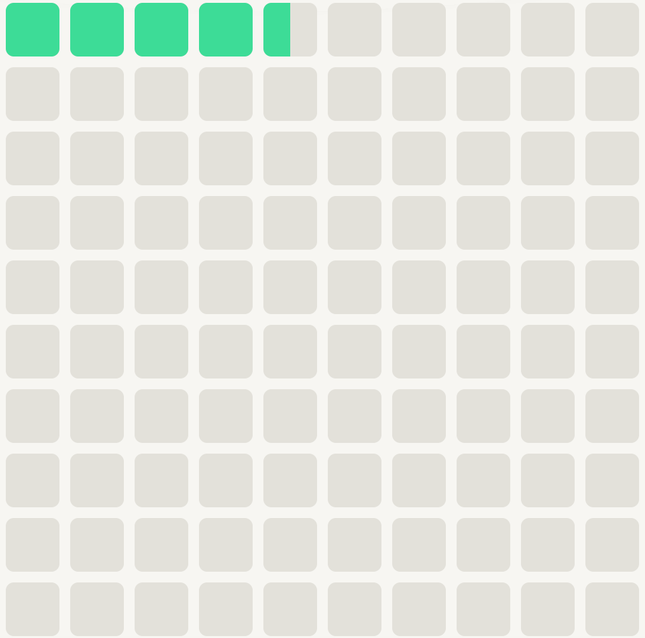
Fresh graduates & juniors

What does ‘job-ready’ mean now?

04

Everything from Stop 03 applies — now the job-market view.

The entry door is narrowing.



4.5% of software-development postings are entry-level · 69.3% are senior

Indeed Hiring Lab · US postings · Q1 2026

New-grad hiring at the largest tech companies: about 65% below 2019

SignalFire State of Talent · Jun 2026

Sources: Indeed Hiring Lab, Jul 2026 · SignalFire, Jun 2026

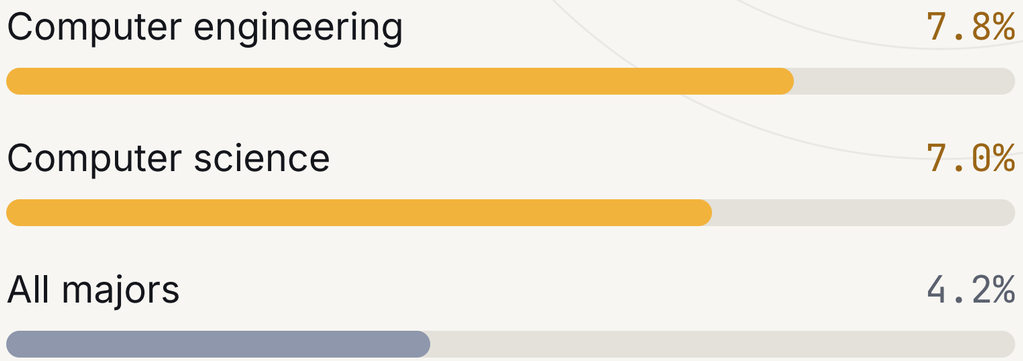
The entry door is narrowing.

19%

below the path of less-exposed peers — employment of 22–25-year-olds in AI-exposed jobs

Stanford Digital Economy Lab · ADP payroll data to Jun 2026 · Aug 2026

RECENT-GRADUATE UNEMPLOYMENT (AGES 22–27)



Federal Reserve Bank of New York · 2024 data, published Feb 2026

Sources: Stanford Digital Economy Lab, Aug 2026 · Federal Reserve Bank of New York, Feb 2026

ITWORX SUMMER INTERNSHIP 2026 · WORXFORGE

Good news: juniors can build real software

Four students. Four weeks. A new stack. One working application.

A locally deployed .NET and React application for publishing, discovering and reusing knowledge assets.

Delegate	Coding agents generated the application code
Direct	The students directed the agents
Validate	They tested the results and iterated toward the agreed behaviour
Own	The students owned the challenge — with technical mentorship

One observational case with technical mentorship, not a controlled study.

Here's the code.

github.com/itworx/WorxForge

Open source (MIT) · whitepaper in the repo

Sources: ITWorx · WorxForge internship case, 2026

THE NEW BASELINE

Yesterday's senior skills are today's junior foundation.

WAS SENIOR		NOW DAY ONE
Reviewing other people's code	→	**Reviewing an agent's diff**
Writing a spec for the team	→	**Writing context and acceptance criteria for an agent**
Debugging unfamiliar code	→	**Debugging code nobody on the team typed**
Thinking about failure modes	→	**A negative test with every change**
Explaining trade-offs	→	**Defending every line of your PR**

16% of senior developers say juniors fully understand the AI code they submit

BairesDev Dev Barometer Q2 2026 · 510 senior developers · vendor survey

Sources: BairesDev, Jun 2026

The new entry-level bar.

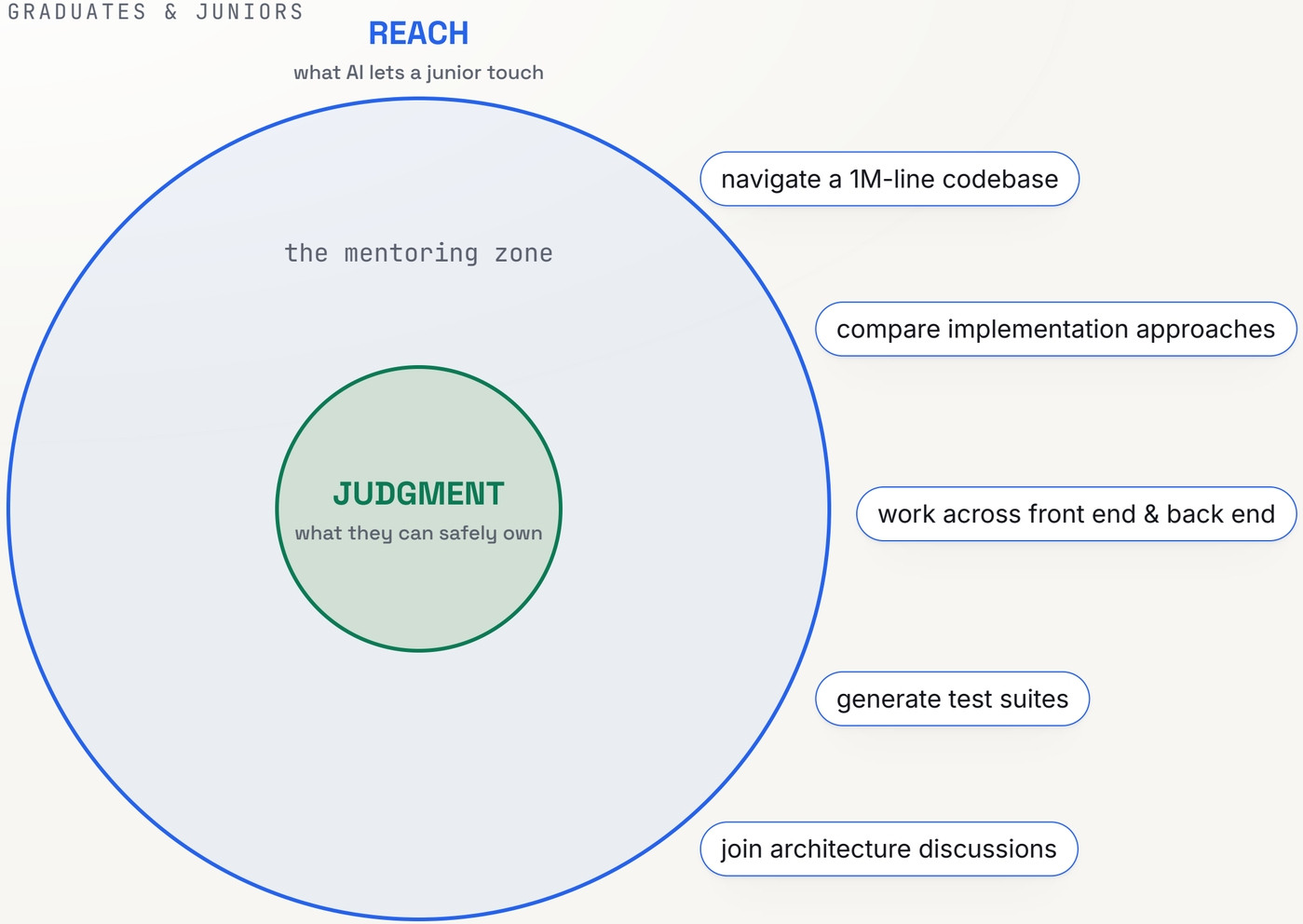
KNOW	DS&A · HTTP & APIs · SQL · auth & OWASP basics · Git
BUILD BY HAND	a small service with AI off · read a stack trace
BUILD WITH AI	ship a feature in a real repo with an agent
DEBUG	find the cause, not just the fix
VALIDATE	find the bug in an AI diff · write the negative test
EXPLAIN	every line of their PR
NOT YET	system architecture — literacy only

64% of senior developers: top graduate mistake is ‘over-reliance on AI without understanding’ — BairesDev Q2 2026 (vendor survey)

Build it with AI. Explain it without AI.

Sources: BairesDev Q2 2026

2026



AI gives juniors senior reach — not senior judgment.

AI can expand your reach faster than your judgment develops.

YOUR PLAN · FRESH GRADUATES & JUNIORS

Your first 90 days.

Stand out

BEFORE DAY ONE

- A project with a process log
- One merged open-source PR
- An “AI mistakes I caught” write-up
- Explain every line
- Volunteer for cross-stack tickets — breadth, on the job

Days 1–30

LEARN THE SYSTEM

- Set up the agent with the team's context files
- Read the codebase with AI as a guide
- Fix three small bugs — first attempt by hand

Days 31–60

CONTRIBUTE

- Issue → plan → agent → verify → PR
- A negative test with every change
- Pair with a senior on one agent session

Days 61–90

OWN

- One small feature, end to end
- A written design rationale
- Catch one bug before review

> Daily AI users reached their 10th pull request in 49 days, vs 91 days without AI (DX, six enterprises).

EXAMPLE TOOLS — NOT ENDORSEMENTS

UNDERSTAND Copilot Chat · Cursor · Claude Code **VERIFY** your test framework · Playwright **REVIEW** the team's pull-request review

ACTION **Make it visible: every PR explained, every agent mistake logged.**

Sources: DX, Sep 2025

STOP 05

Experienced software engineers

How do working engineers keep growing?

05

The job changed. The skills model must follow.

2026

Conceptual — not measured data



“...AI is writing 90 percent of the code.”

“But the programmer still needs to specify... what is the overall app you're trying to make, what's the overall design decision?”

— Dario Amodei, CEO, Anthropic · Council on Foreign Relations · 10 Mar 2025 · a forecast: “in three to six months”

Sources: Dario Amodei, Council on Foreign Relations, 10 Mar 2025

AI lowers the cost of learning a new technology — and raises the cost of staying narrow.

~~vibe coding~~
Karpathy, Feb 2025: "...forget that the code even exists."

→

agentic engineering
Karpathy, Feb 2026: "...orchestrating agents who do and acting as oversight"

L5 Systematising
team standards, hooks, guardrails, evaluation

L4 Orchestrating
parallel agents + context files you maintain

L3 Delegating
issue → approved plan → PR, with your own verification

L2 Paired
agent in the editor; review every change

L1 Assisted
ask for explanations, never code

Frame → Context → Plan → Delegate → Verify → Own

AGENTS.MD — SHOP-API

```
# Auth: JWT bearer. ALWAYS validate issuer, audience, lifetime, signing key
# Every auth change needs negative tests
# Plan first; wait for approval before editing more than 3 files
# Never commit secrets
```

Practise with a real starter repo: github.com/hanygheit/ai-agent-on-call

Agentic coding is learned, not prompted.

Sources: Karpathy, Feb 2025 · Karpathy, Feb 2026

1 Student | 2 Fresh grad | 3 Junior | 4 Mid | 5 Senior | 6 Lead/Architect

Full blueprint → /blueprint

radius · feature slice

	Student	Fresh grad	Junior	Mid	Senior	Lead
Foundations	5	5	4	3	2	2
Manual implementation	5	4	3	2	1	1
Debugging & testing	4	5	5	4	3	3
Agentic coding & context	2	3	4	5	5	4
AI validation	3	4	5	5	5	4
Systems & architecture	1	2	3	4	5	5
Product & communication	2	3	3	4	4	5
Judgment & ownership	2	2	3	4	5	5

Proposed learning emphasis, not skill scores

YOUR NEXT 90 DAYS

- Issue→plan→agent→verify→PR daily
- Weekly AI-off kata
- Own regression tests

NEVER DELEGATE

acceptance criteria and final review

READY FOR NEXT LEVEL WHEN…

you catch agent errors before reviewers do

Seniority is measured by what you can safely delegate — and still own.

YOUR LEARNING SYSTEM · EXPERIENCED SOFTWARE ENGINEERS

New ways to grow — at every level.

Pair

WITH AN AGENT

- State the intent, approve the plan
- Review every diff
- Use it as a tutor on unfamiliar code

Practise

AI-OFF

- One kata or debugging session a week
- Keep the muscle you verify with

Review

AGENT SESSIONS

- Plans, prompts and diffs, reviewed as a team
- Seniors walk juniors through theirs

Write

THE CONTEXT

- AGENTS.md, ADRs, guardrails
- What every agent on the team reads

> Frontier professionals are more likely to do some work without AI to keep their skills sharp — 43% vs 30% (Microsoft Work Trend Index 2026).

BY LEVEL

DEVELOPER Master one agent end to end · verify before review

SENIOR Own the guardrails · threat-model AI changes · coach agent sessions

LEAD Team AI standards · competency matrix · measure stability

Example tools, not endorsements: Claude Code · OpenAI Codex · Copilot coding agent · Gemini CLI · AI pull-request review

ACTION **This sprint: review one agent session as a team — and do one task AI-off.**

Sources: Microsoft Work Trend Index, May 2026

STOP 06

Software companies & teams

06

What must organisations start now?

01 The shift
02 Universities & educators
03 Students
04 Fresh graduates & juniors
05 Experienced software engineers
06 **Software companies & teams**
07 Software buyers & stakeholders

SOFTWARE COMPANIES & TEAMS

AI adoption is no longer optional.

> “Reflexive AI usage is now a baseline expectation at Shopify”

Tobi Lütke
CEO, Shopify
Internal memo, published on X · 7 Apr 2025

> “...using AI is no longer optional — it's core to every role and every level.”

Julia Liuson
Developer Division, Microsoft
Internal memo, via Business Insider · 27 Jun 2025

90% of technology professionals already use AI at work
DORA 2025

72% say the skills expected of them have shifted — only 36% feel adequately upskilled
BCG 2026

No longer optional — but a licence is not a strategy.

Sources: Tobi Lütke on X, Apr 2025 · Business Insider, Jun 2025 · DORA, Sep 2025 · BCG, Jun 2026

100% licensed. Not necessarily AI-ready.

Tool-first

- Buy licences
- One-off workshop
- Count active users
- Assume productivity improved

Capability-first

- ✓ Choose workflows & safeguards
- ✓ Train & mentor by career stage
- ✓ Evaluate delivery & quality
- ✓ Scale on demonstrated outcomes

100% licensed. Not necessarily AI-ready.

JUNIOR · CODE QUALITY

BEFORE

Writes clean, readable code that follows team standards and is covered by unit tests.

AFTER

Delivers changes — by hand or with an agent — that meet team standards; can **explain** every line of their PR; adds the **test** that would catch the agent's likely mistake; **flags** any AI output they cannot verify.

	BEFORE	AFTER
Onboarding	Read docs, fix a starter bug	First feature with an agent + walkthrough · AI-off debugging drill · team context files
Development plan	“Complete an AI course”	“Deliver one agent-assisted repository-level change, show the context and constraints you gave the agent, and independently validate the result against defined acceptance criteria.”
Mentoring	Answer questions	Review agent sessions (plan, prompts, diffs) · explain-it-back
Promotion	Output volume	Delegation radius · decision quality · defects caught before review · people grown

100% licensed. Not necessarily AI-ready.

01 Time to 10th PR: 91 → 49 days for daily AI users (DX, industry data)

02 ‘the first stop for questions that used to go to colleagues’ — Anthropic, Dec 2025

03 ‘manage juniors for learning, not production.’ — Kent Beck

Sources: DX · Anthropic, Dec 2025 · Kent Beck

Business value	outcomes delivered, cost per change	OUTCOMES
Quality	change failure rate, escaped defects, incident recovery	
Delivery	lead time, review turnaround	
Capability	agentic maturity, review quality, defects caught before review	
Adoption	seats, usage, % AI-generated code	ACTIVITY

Faros, Sep 2026: PRs merged with no review +76%

DORA: throughput up — and instability up

McKinsey: 80% report gains · only 37% see EBIT

Don’t measure how much AI they use. Measure what they can safely own.

AI scales execution. Our job is to scale engineering capability.

Sources: Faros AI, Sep 2026 · DORA, Mar 2026 · McKinsey, Aug 2026

YOUR 90-DAY ROADMAP · SOFTWARE COMPANIES & TEAMS

Adopt, train and measure — together.

Days 1–30

FOUNDATIONS

- A clear AI stance and guardrails: data, security, review rules
- Two or three workflows, with pilot teams
- Baseline delivery and quality before rollout
- Context files in key repositories

Days 31–60

CAPABILITY

- Training by career stage — not one workshop
- Mentoring through agent-session reviews
- Update onboarding and one competency row

Days 61–90

MEASURE & SCALE

- Compare with the baseline: delivery, quality, capability — not seats
- Scale or stop each workflow
- Update development plans and promotion criteria

DORA's AI Capabilities Model: clear AI stance · healthy data ecosystems · AI-accessible internal data · strong version control · small batches · user-centric focus · quality internal platforms. BCG: regular use is sharply higher with at least five hours of training.

ACTION **Start Monday: baseline first — then adopt, train and measure together.**

Sources: DORA AI Capabilities Model, Sep 2025 · BCG, Jun 2025

STOP 07

Software buyers & stakeholders

07

What should you expect from AI-built software?

A WORD FOR SOFTWARE BUYERS & STAKEHOLDERS

Faster — not free, and not instant.

Yes, you can build simple apps yourself.

Prototypes, internal tools and automations are now within reach — with the builder tools from Stop 01.

Agentic AI helps teams deliver faster and at lower cost.

Delivery cycles shrink when engineers direct agents well.

It's still engineering.

Security, integration, data, scale and support take expertise, time and budget.

ACTION **Your move: ask your vendor how they use AI — and how they verify it.**

1 Student | 2 Fresh grad | 3 Junior | 4 Mid | 5 Senior | 6 Lead

A responsibility framework, not a sequence

DELEGATE

Which bounded execution can AI perform?

Well-specified implementation

DIRECT

What intent, context, constraints and acceptance criteria must I set?

Plan and context for a feature slice

VALIDATE

What evidence must I inspect, test and challenge?

Edge cases, tests, basic security

OWN

What am I accountable for, whatever wrote it?

Their PRs to production

ENGINEER OF RECORD

TEACH universities

PREPARE graduates

DEVELOP engineers

ENABLE software companies & teams

Student → Graduate → Junior → Mid → Senior → Lead

Where should we raise the engineering bar first?

WHO'S IN THE ROOM

Live results appear here during the talk

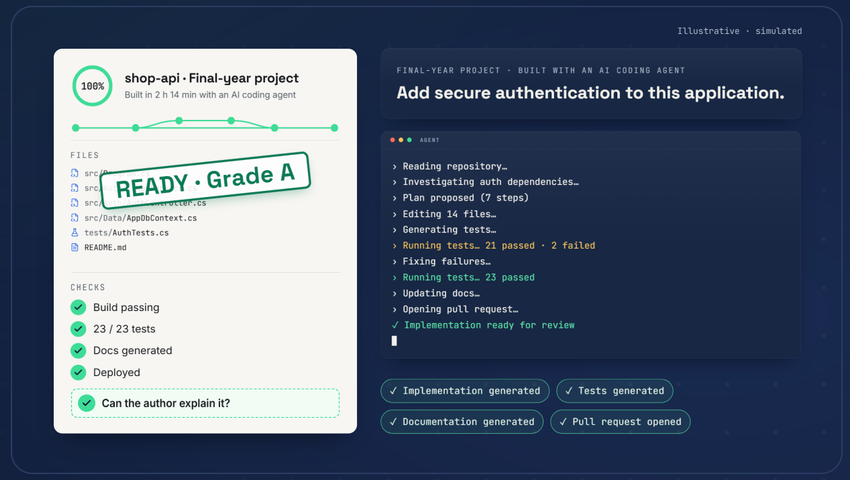
1. **Learning** — student or recent graduate
2. **Building** — developer or engineer
3. **Teaching** — educator, professor or trainer
4. **Leading** — tech lead, manager or company leader

Hold your questions — Q&A right after the close.

Feedback · slides · PDF · code — all in one place

brainsmingle-ai-era-software-engineering.lovable.app/thanks

DELEGATE DIRECT VALIDATE OWN



The code is ready. Are we?

Yes — we can be.

ONE LINE FOR EACH OF US

EDUCATORS

Change one assignment: AI allowed, explanation required.

STUDENTS

Learn it by hand, then with AI — and explain both.

GRADUATES

Show evidence, not demos: every PR explained, every agent mistake caught.

EXPERIENCED SOFTWARE ENGINEERS

Grow your delegation radius — and review one agent session a week.

LEADERS

Rewrite one development plan around what people can safely own.

SOFTWARE COMPANIES & TEAMS

Start the 90 days: baseline, train by stage, measure outcomes — not usage.

SOFTWARE BUYERS & STAKEHOLDERS

Expect faster, not free: ask how it's built and how it's verified.

AI can write the code. Engineers still own the outcome.

AI is changing how we build software.
We must change how we develop software engineers.

ONE CHANGE · NEXT 7 DAYS

Write down one change you'll make in the next seven days.

DELEGATE	DIRECT	VALIDATE	OWN
bounded execution	intent, context, constraints	correctness, quality, risk	decisions, delivery, outcome

Q&A · next 10 minutes — ask in the Q&A section

Feedback · slides · PDF · code — all in one place

brainsmingle-ai-era-software-engineering.lovable.app/thanks

Hany Saad
Senior Software Engineering Manager · ITWorx

RESOURCES

AI-Era Software Engineering — The Blueprint

Hany Saad · Senior Software Engineering Manager · ITWorx

(A)

Plans by role — the action behind every stop

Every stop of the talk ends with one action. These are the full plans behind them. Tools are named as examples, grouped by purpose — not endorsements.

Universities & educators

ACTION **This term: change one assignment — AI allowed, explanation required.**

THIS MONTH — SET THE RULE

- Publish an AI rule for every assignment: a secure lane (AI-off, in person or oral) or an open lane (AI allowed and scaffolded) — e.g. the University of Sydney's two-lane model
- Or label each task with the five-level AI Assessment Scale: No AI · AI Planning · AI Collaboration · Full AI · AI Exploration
- Every lecturer runs a coding agent on their own assignment and notes what it gets right and wrong

THIS TERM — CHANGE THE EVIDENCE

- Pick one assignment per course and add a 5-minute oral defence, a planted-bug review, a live change request (15 minutes, AI allowed, explained afterwards) and a process log
- Use the rubric in section (e) and calibrate it across staff with three sample submissions
- Teach the Keep · Shift · Add · Prove model explicitly to students

THIS YEAR — CHANGE THE COURSE

- AI-off checkpoints in foundation courses; an agentic capstone in the final year
- A monthly staff community of practice: share assignments, rubrics and failures
- Compare grades with oral-defence results to see where the old assessment over-graded

Students

ACTION **One project: build it by hand, extend it with an agent, explain every line.**

THE MYTH — "AI DOES EVERYTHING, SO I DON'T NEED TO LEARN"

- Wrong. Less of: memorising syntax, typing boilerplate, hunting through API docs
- More of: engineering — design, debugging, testing, security, reading & reviewing code
- You still read and understand every line — and write it yourself when needed
- Good news: learning a new concept has never been faster. The catch: there's more to learn

FOUNDATIONS — THE ENGINEERING CORE

- Know how an OS runs your code: processes, memory, threads, files
- Model data and write real SQL: schemas, joins, indexes, transactions
- Design with OOP — not just classes: responsibilities, interfaces, composition
- Solve the problem before you prompt: decompose it and sketch it, then ask
- Programming, data structures & algorithms; HTTP & APIs; Git, testing and debugging; auth basics and the OWASP Top 10
- See the whole SDLC: requirements → design → build → test → deploy → operate

BREADTH — GO BROAD, NOT NARROW

- Update the syllabus: keep the core, shift the weight, add agentic workflows and AI-aware security

- Be a product engineer — own a feature end to end: the user problem, UI, API, data, tests, release. It's a way of working with product and design; it doesn't replace analysts or testers
- Learn a business domain (fintech, health, e-commerce…): domain understanding is what AI can't supply
- Keep one deep strength (T-shaped), but don't define yourself by a single layer or framework

EDGE — WHAT SETS YOU APART

- Technical English — a prompt is a specification; write it clearly in English: inputs, outputs, constraints, acceptance criteria
- GenAI & agents — after you can code: by hand → with AI → with agents. Agentic coding is a skill too — learn it after you've written code yourself
- Read code you didn't write — and review AI output like a teammate's pull request

LEARNING ORDER

- 1 · Write it yourself
- 2 · Use AI to explain, compare and critique
- 3 · Delegate to agents — once you can verify the result

WEEKLY HABITS

- First version by hand — AI may explain, not solve
- Log what the AI got wrong — and how you noticed
- Teach back one concept a week: a post, a talk or a study partner
- A 30-minute AI-off kata: an algorithm, a bug or a small refactor

CONCEPTS OUTLIVE TOOLS — LEARN THE CONCEPT BEHIND EACH TOOL

- Framework APIs → separation of concerns

- ORM syntax → data modelling & transactions
- AI coding products → decomposition & validation
- Cloud commands → scalability & reliability
- Security libraries → authN/authZ & threat modelling
- Frameworks change. Engineering principles travel with you.

PORTFOLIO PROJECT — ONE SMALL API, BUILT TWICE

- v1 by hand: a small REST API with auth, a database and tests
- v2: extend it with a coding agent — one new feature, end to end
- Ship a README, an AGENTS.md (your rules for the agent) and a process log
- Negative tests for every auth rule, and a short “3 AI mistakes I caught” write-up
- A 5-minute walkthrough video that explains every design decision

READY WHEN

- You can build and debug a small service with AI off — and explain the version you built with AI

Fresh graduates & juniors

ACTION **Make it visible: every PR explained, every agent mistake logged.**

STAND OUT — BEFORE DAY ONE

- One project with a process log and an AGENTS.md
- One merged open-source pull request
- An “AI mistakes I caught” write-up
- Be ready to explain every line — and to change it live

BREADTH ON THE JOB — EVERYTHING FROM STOP 03 APPLIES

- The Students plan is yours too: foundations, breadth (the product-engineer mindset) and edge
- Volunteer for cross-stack tickets — breadth, on the job
- Learn the business domain of your first product: ask why each feature exists

DAYS 1-30 — LEARN THE SYSTEM

- Set up your agent with the team's context files and conventions
- Read the codebase with AI as a guide; draw the architecture yourself
- Fix three small bugs — first attempt by hand, then compare with the agent

DAYS 31-60 — CONTRIBUTE

- Issue → plan → agent → verify → PR, with the plan approved before the code
- A negative test with every change

Experienced software engineers

ACTION **This sprint: review one agent session as a team — and do one task AI-off.**

DEVELOPER

- Master one agent end to end: issue → approved plan → PR, with your own verification
- A weekly AI-off kata; own the regression tests
- Ready when you catch agent errors before reviewers do

SENIOR

- Own the guardrails and the context: AGENTS.md, hooks, review rules
- Threat-model AI-generated changes
- Coach juniors through their agent sessions — plans, prompts and diffs
- Ready when the teams around you produce better AI-assisted work

LEAD / ARCHITECT

- Publish team AI standards and update the competency matrix
- Measure stability as well as throughput
- Make agent-session review part of mentoring and promotion
- Ready when the team ships faster, stability holds and juniors grow judgment

NEW WAYS OF LEARNING — EVERY LEVEL

- Pair with an agent: state the intent, approve the plan, review every diff
- Practise AI-off: one kata or debugging session a week
- Review agent sessions as a team, like code review

- Pair with a senior on one agent session; ask them to review your prompts and diffs

DAYS 61–90 — OWN

- One small feature, end to end
- A written design rationale: two alternatives considered, and why this one
- Catch one bug before review — and write down how you found it

- Write the context every agent on the team reads

Engineering leaders

ACTION **Rewrite one development plan around what people can safely own.**

DEVELOPMENT PLANS

- Replace “complete an AI course” with a demonstrable outcome: deliver one agent-assisted repository-level change, show the context and constraints you gave the agent, and independently validate the result against defined acceptance criteria

MENTORING

- Review agent sessions — plan, prompts, diffs — and ask people to explain the change back
- Manage juniors for learning, not production (Kent Beck)

PROMOTION

- Delegation radius · decision quality · defects caught before review · people grown — not output volume

Software companies & teams

ACTION **Start Monday: baseline first — then adopt, train and measure together.**

DAYS 1-30 — FOUNDATIONS

- A clear, communicated AI stance and guardrails: data, security, review rules
- Two or three workflows, with pilot teams
- Baseline delivery and quality (lead time, change failure rate, escaped defects) before rollout
- Context files in key repositories

DAYS 31-60 — CAPABILITY

- Training by career stage, not a one-off workshop — BCG: regular use is sharply higher with at least five hours of training
- Mentoring through agent-session reviews
- Update onboarding and one competency row

DAYS 61-90 — MEASURE & SCALE

- Compare with the baseline across delivery, quality and capability — not seats or % AI-generated code
- Scale or stop each workflow on evidence
- Update development plans and promotion criteria

USE AS A CHECKLIST

- DORA AI Capabilities Model: clear AI stance · healthy data ecosystems · AI-accessible internal data · strong version control · small batches · user-centric focus · quality internal platforms

Software buyers & stakeholders

ACTION **Ask your vendor how they use AI — and how they verify it.**

WITHIN YOUR REACH

- Simple apps you build yourself: prototypes, internal tools and automations

WHAT AGENTIC AI CHANGES

- Teams deliver faster and at lower cost — delivery cycles shrink when engineers direct agents well

WHAT IT DOESN'T CHANGE

- It's still engineering: security, integration, data, scale and support take expertise, time and budget

Example tools by purpose — not endorsements

LEARNING	ChatGPT study mode · Claude learning mode · Gemini Guided Learning · CS50.ai
CODING ASSISTANTS	GitHub Copilot (free for verified students) · Cursor · JetBrains AI · Gemini Code Assist
CODING AGENTS	Claude Code · OpenAI Codex · Copilot coding agent · Gemini CLI

TESTING & REVIEW	xUnit / JUnit / pytest · Playwright · AI pull-request review

(B)

The full 16-capability emphasis matrix

CAPABILITY	STUDENT	FRESH GRAD	JUNIOR	MID	SENIOR	LEAD
Foundations	5	5	4	3	2	2
Manual implementation	5	4	3	2	1	1
Debugging	4	5	5	4	3	2
Testing	3	4	5	4	3	3
Problem decomposition	3	3	4	5	5	4
AI as tutor	4	4	3	2	2	2
Agentic coding	2	3	4	5	4	3
Context engineering	1	2	3	4	5	5
AI validation	3	4	5	5	5	4
System thinking	1	2	3	4	5	5
Architecture	1	1	2	3	5	5
Security	2	3	3	4	5	4
Product/domain	1	2	3	4	4	5

CAPABILITY	STUDENT	FRESH GRAD	JUNIOR	MID	SENIOR	LEAD
Engineering judgment	2	2	3	4	5	5
Communication	2	3	3	4	4	5
Ownership	2	3	4	4	5	5

Proposed learning emphasis, not skill scores.

(C)

The six stage plans

Student

Delegation radius: snippet

YOUR NEXT 90 DAYS

- Build a small API by hand and explain it
- Extend it with an agent and log what it got wrong
- Teach back one concept a week
- Go broad, not narrow: be a product engineer, not just a “frontend” or “backend” developer

NEVER DELEGATE

the first version of anything you're learning

READY FOR NEXT LEVEL WHEN…

you can build and debug a small service with AI off

Fresh grad

Delegation radius: function

YOUR NEXT 90 DAYS

- Master one coding agent end to end
- Ship one project with a process log
- Practise debugging unfamiliar code
- Go broad, not narrow: be a product engineer, not just a “frontend” or “backend” developer

NEVER DELEGATE

understanding why the code works

READY FOR NEXT LEVEL WHEN…

you take an issue to a merged PR and walk a reviewer through it

Junior

Delegation radius: feature slice

YOUR NEXT 90 DAYS

- Issue→plan→agent→verify→PR daily
- Weekly AI-off kata
- Own regression tests

NEVER DELEGATE

acceptance criteria and final review

READY FOR NEXT LEVEL WHEN…

you catch agent errors before reviewers do

Mid

Delegation radius: feature / service

YOUR NEXT 90 DAYS

- Decompose an epic into agent-sized tasks
- Maintain repo context files
- Write two ADRs

NEVER DELEGATE

decomposition and interface contracts

READY FOR NEXT LEVEL WHEN…

you deliver ambiguous work safely through agents and people

Senior

Delegation radius: subsystem

YOUR NEXT 90 DAYS

- Own guardrails & context engineering
- Threat-model AI changes
- Mentor juniors on agent sessions

NEVER DELEGATE

architecture and security trade-offs

READY FOR NEXT LEVEL WHEN…

teams around you produce better AI-assisted work

Lead/Architect

Delegation radius: team workflow

YOUR NEXT 90 DAYS

- Publish team AI standards
- Update the competency matrix
- Measure stability as well as throughput

NEVER DELEGATE

accountability and people decisions

READY FOR NEXT LEVEL WHEN…

the team ships faster, stability holds, juniors grow judgment

(D)

The 8-week graduate plan

Weeks 1–2

Fundamentals by hand — a small API built AI-off, explained.

Weeks 3–4

One agent end to end — extend the API into a feature that spans UI, API and data, with a process log.

Weeks 5–6

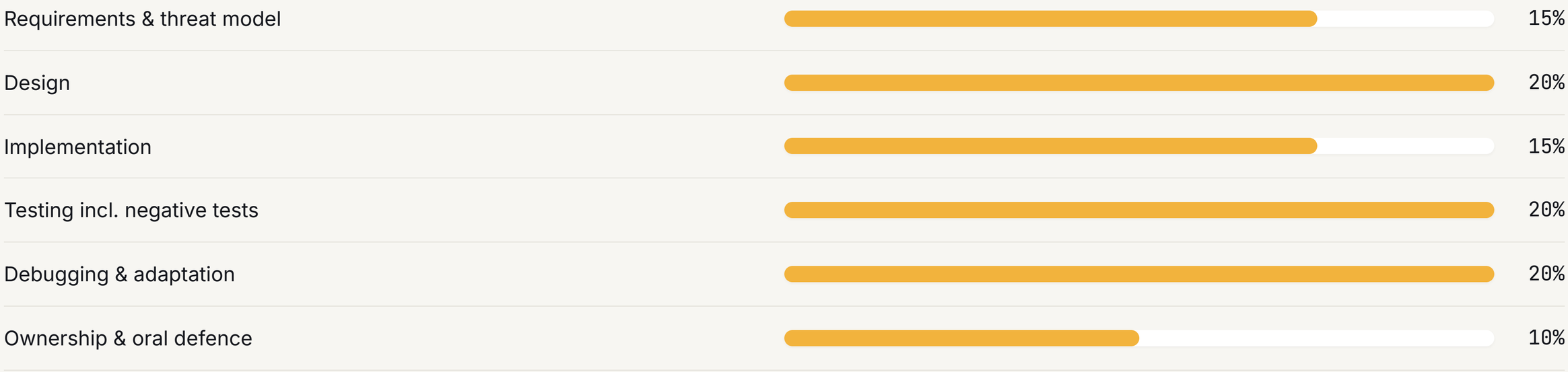
Validation — an “AI mistakes I caught” write-up.

Weeks 7–8

Change and explain — a design rationale and a mock debugging interview.

(E)

The weighted assessment rubric



~~Did the code work?~~

Can the student explain why it works?

Oral walkthrough, 5 minutes: trace one request through the code, end to end.

~~Are tests passing?~~

Can they say what the tests miss?

Name one case the tests don't cover — then write that test.

~~Is it implemented?~~

Can they change it when the requirement changes?

Live modification: a new requirement, 15 minutes, AI allowed, explained afterwards.

~~Was it submitted?~~

Can they defend the decisions?

A short design rationale: two alternatives considered, and why this one.

(F)

The five-layer scorecard

1	**Adoption**	seats, usage, % AI-generated code	ACTIVITY
2	**Capability**	agentic maturity, review quality, defects caught before review	
3	**Delivery**	lead time, review turnaround	
4	**Quality**	change failure rate, escaped defects, incident recovery	
5	**Business value**	outcomes delivered, cost per change	OUTCOMES

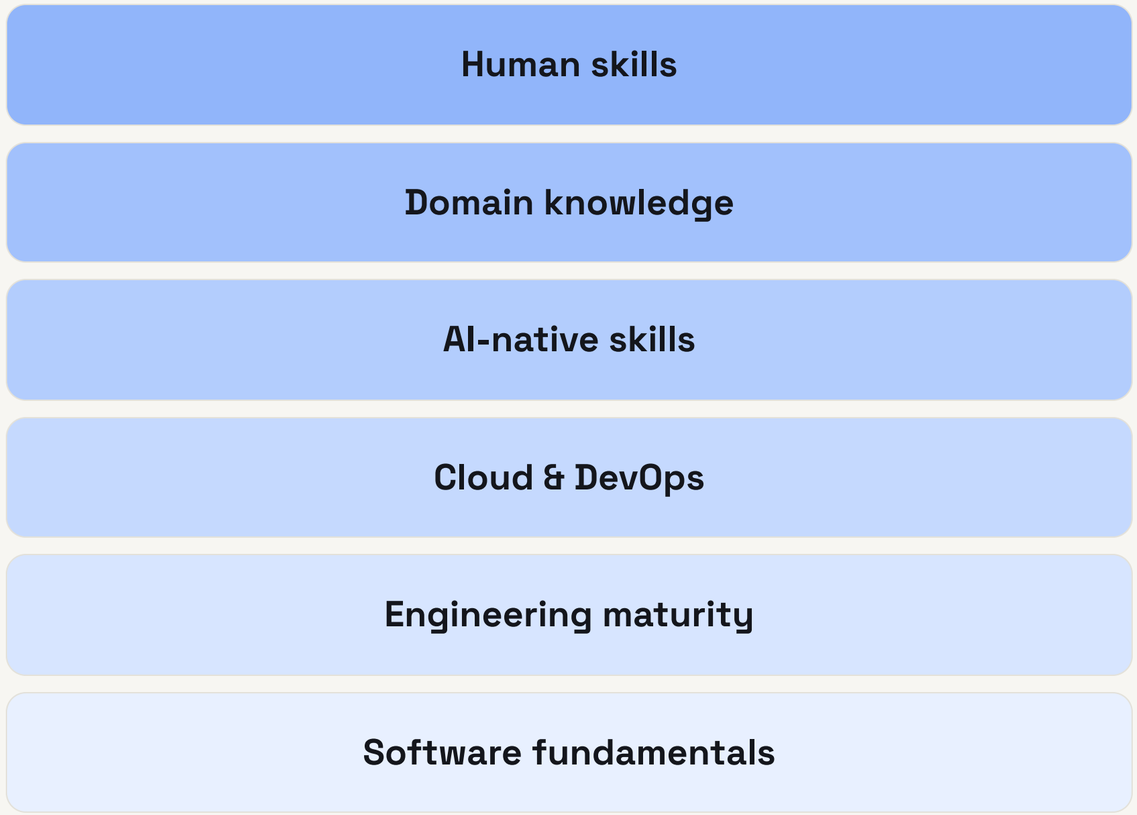
(G)

Delegate / Direct / Validate / Own by stage

STAGE	DELEGATE	DIRECT	VALIDATE	OWN
Student	Explanations, hints, examples	Own learning goal: explain, don't solve	Check explanations against docs	Their understanding and submission
Fresh grad	Scaffolding, exploration, test drafts	Acceptance criteria for a small task	Every line of the diff	One function or fix end to end
Junior	Well-specified implementation	Plan and context for a feature slice	Edge cases, tests, basic security	Their PRs to production
Mid	Multi-step & parallel implementation	Decomposition, interfaces, context files	Verification strategy; others' AI PRs	A feature or service
Senior	Most implementation; investigations; prototypes	Architecture, guardrails, conventions	Threat models; quality bar	A subsystem and its reliability
Lead	Analysis, drafting, first-pass review	Team AI standards, SDLC, capability framework	Quality gates, metrics, stability	Team outcomes and people's growth

(H)

The layered learning stack



Four soft skills that beat any framework

- Precise specification & communication
- Problem decomposition
- Verification & critical reading
- Product & business judgment

(I)

Starter repo for agentic practice

github.com/hanygheit/ai-agent-on-call

AGENTS.md, hooks and tests to practise L3–L5 delegation.

(J)

All sources

S5 · EVERYONE CAN BUILD SOFTWARE NOW

Lovable hits $200M ARR — Fortune (Beatrice Nolan), 21 Nov 2025

Anton Osika on everyone becoming a developer — Fortune interview with Anton Osika (Lovable CEO), 21 Nov 2025

2026 Developer Survey — coding agents — Stack Overflow, 6 Oct 2026

S6 · THE UNIT OF DELEGATION CHANGED

Dependabot alerts are now assignable to AI agents for remediation — GitHub Changelog, 7 Apr 2026

S8 · AI IS ALREADY INSIDE THE WORKFLOW

Alphabet Q3 earnings call: CEO Sundar Pichai's remarks — Google, 29 Oct 2024

Sundar Pichai shares news from Google Cloud Next 2026 — Google & Alphabet, 22 Apr 2026

When AI builds itself — Anthropic Institute (Marina Favaro, Jack Clark), 4 Jun 2026 (updated 18 Sep 2026)

Anthropic's viral new work tool wrote itself — Axios (Megan Morrone), 13 Jan 2026

AI Coding Agents: Adoption Trends — JetBrains Research (Mikhail Bogdanov), 18 Aug 2026

2026 Developer Survey — frequency of AI use — Stack Overflow, 6 Oct 2026

S9 · DEVELOPERS INCREASINGLY DIRECT, REVIEW AND VALIDATE

Sundar Pichai shares news from Google Cloud Next 2026 — Google & Alphabet, 22 Apr 2026

Agentic engineering (post) — Andrej Karpathy on X, 4 Feb 2026

Developers, Reinvented — Thomas Dohmke (then GitHub CEO) & Eirini Kalliamvakou, 3 Aug 2025

2026 Developer Survey — trust in AI — Stack Overflow, 6 Oct 2026

S10 · WHEN WRITING CODE IS NO LONGER THE CONSTRAINT

Agentic coding is straining CI: how we scaled test impact analysis at Anthropic — Anthropic (Sachin Malhotra), 14 Sep 2026

S11 · MORE CODE IS NOT MORE ENGINEERING VALUE

Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity — METR, 10 Jul 2025

We are Changing our Developer Productivity Experiment Design — METR, 24 Feb 2026

The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers — Cui, Demirer, Jaffe, Musolff, Peng, Salz — Management Science, Published online 27 Feb 2026

The Speed Trap: 8 takeaways from the AI Engineering Report — Faros AI, 18 Sep 2026

Balancing AI tensions — DORA (Jessica Baolin, Nathen Harvey), 10 Mar 2026

2025 Developer Survey — AI — Stack Overflow, Jul 2025

S13 · YOUR MOVE · NEVER DELEGATE WHAT YOU CAN'T VERIFY

2025 Developer Survey — AI — Stack Overflow, Jul 2025

S17 · AI-NATIVE IS NOT FUNDAMENTALS-FREE · DOHMKE

Developers, Reinvented — Thomas Dohmke (then GitHub CEO) & Eirini Kalliamvakou, 3 Aug 2025

S18 · IF AI CAN BUILD IT, WHAT ARE WE ASSESSING?

Designing AI-resistant technical evaluations — Anthropic (Tristan Hume), 21 Jan 2026

What faculty want: key results from the Global AI Faculty Survey 2025 — Digital Education Council, 28 Jan 2025

S19 · YOUR ROLLOUT · CHANGE ONE ASSIGNMENT THIS TERM

Student Generative AI Survey 2026 (HEPI Report 199) — HEPI & Kortext (Rose Stephenson, Charlotte Armstrong), 12 Mar 2026

Frequently asked questions about the two-lane approach to assessment in the age of AI — University of Sydney (Adam Bridgeman, Danny Liu), 2 Jul 2024, updated 21 Apr 2026

The AI Assessment Scale (AIAS) — Perkins, Furze, Roe & MacVaugh, 2024; revised 2025

Purdue's AI competency graduation requirement begins in every undergraduate degree program — Purdue University, 19 Aug 2026

S21 · THE MYTH · AI DOES EVERYTHING, SO I DON'T NEED TO LEARN

The Batch, Issue 292 — letter — Andrew Ng (DeepLearning.AI), 12 Mar 2025

S22 · LEARN WITH AI — AND WITHOUT IT

Balancing AI tensions — DORA (Jessica Baolin, Nathen Harvey), 10 Mar 2026

How AI assistance impacts the formation of coding skills — Anthropic (Judy Hanwen Shen, Alex Tamkin), 29 Jan 2026

A new wave of education: how is AI shifting classroom policies? — The Stanford Daily (Sterling Davies), 7 Mar 2026

S23 · YOUR PLAN · LEARN WITH AI, PROVE IT WITHOUT AI

The Impact of Generative AI on Critical Thinking: Self-Reported Reductions in Cognitive Effort and Confidence Effects From a Survey of Knowledge Workers — Microsoft Research & Carnegie Mellon (Lee et al.), CHI 2025, Apr 2025

Setting up Copilot for students — GitHub Docs, 2026

What is a product engineer (and why they're awesome) — PostHog (Ian Vanagas), 1 Dec 2022

S25 · THE ENTRY DOOR IS NARROWING

The labor market is tilting toward seniority — Indeed Hiring Lab (Felix Aidala, Sneha Puri), 23 Jul 2026 (Q1 2026 data)

SignalFire State of Talent Report 2026 — SignalFire, 22 Jun 2026

S26 · THE ENTRY DOOR IS NARROWING · 22–25-YEAR-OLDS

Canaries in the Coal Mine — Stanford Digital Economy Lab (Brynjolfsson, Chandar, Chen), Revised 12 Aug 2026

The Labor Market for Recent College Graduates — Federal Reserve Bank of New York, 2024 data, published 4 Feb 2026

S27 · GOOD NEWS: JUNIORS CAN BUILD REAL SOFTWARE

Juniors Can. Here's the Code. — ITWorx · WorxForge, Summer Internship 2026

S28 · YESTERDAY'S SENIOR SKILLS ARE TODAY'S JUNIOR FOUNDATION

Dev Barometer Q2 2026: the AI career reset — BairesDev, 10 Jun 2026

S29 · THE NEW ENTRY-LEVEL BAR

Dev Barometer Q2 2026: the AI career reset — BairesDev, 10 Jun 2026

S31 · YOUR FIRST 90 DAYS

AI cuts onboarding time in half for new hires in the enterprise — DX (Laura Tacho), 10 Sep 2025

S34 · THE JOB CHANGED · AMODEI

CEO Speaker Series: Dario Amodei — Council on Foreign Relations, 10 Mar 2025

S36 · AGENTIC CODING IS LEARNED, NOT PROMPTED

Agentic engineering (post) — Andrej Karpathy on X, 4 Feb 2026

Vibe coding (post) — Andrej Karpathy on X, 2 Feb 2025

S38 · YOUR LEARNING SYSTEM · NEW WAYS TO GROW

2026 Work Trend Index: agents, human agency and the opportunity for every organization — Microsoft WorkLab, 5 May 2026

S40 · AI ADOPTION IS NO LONGER OPTIONAL

Reflexive AI usage is now a baseline expectation at Shopify — Tobi Lütke (Shopify CEO) on X, 7 Apr 2025

Microsoft internal memo: using AI is no longer optional — Business Insider (Ashley Stewart), 27 Jun 2025

Announcing the 2025 DORA report — Google Cloud (Nathen Harvey, Derek DeBellis), 24 Sep 2025

AI at Work: Strategy Matters More Than Tools — BCG, 3 Jun 2026

S43 · AN AI SUBSCRIPTION IS NOT A DEVELOPMENT STRATEGY · EVIDENCE

AI cuts onboarding time in half for new hires in the enterprise — DX (Laura Tacho), 10 Sep 2025

How AI is transforming work at Anthropic — Anthropic, 2 Dec 2025

The bet on juniors just got better — Kent Beck, 12 Dec 2025

S44 · MEASURE OUTCOMES, NOT LINES OF AI CODE

The Speed Trap: 8 takeaways from the AI Engineering Report — Faros AI, 18 Sep 2026

Balancing AI tensions — DORA (Jessica Baolin, Nathen Harvey), 10 Mar 2026

The state of AI in 2026: On the road to ROI — McKinsey (QuantumBlack), 25 Aug 2026

Introducing DORA's inaugural AI Capabilities Model — Google Cloud (Kevin M. Storer, Derek DeBellis), 24 Sep 2025

AI at Work: Momentum Builds, but Gaps Remain — BCG, 26 Jun 2025



Connect on LinkedIn



Follow on BrainsMingle



This presentation